

vLLM Tutorial

Efficient LLM Inference and Serving

Alexander Conzelmann

September 8, 2026

Goal: Get practical intuition on how GPUs work and how to use vLLM.

- Prefill vs Decode: GPU memory efficiency
- Speculative Decoding: Speeding up inference losslessly

Prefill versus Decode

How GPUs handle memory and computation

Two Distinct Stages in LLM Inference

Due to KV-caching, inference can be separated into **Prefill** and **Decoding**. Let us assume we are working on a single prompt at a time:

Prefill. The tokenized user prompt $P = s_1 s_2 \dots s_m$ is processed. Output is the next token $\hat{s}_{m+1}$ plus per-layer l and per-token k KV cache values $K_{l,k}, V_{l,k}$.

Operations on hidden states X , such as calculating Q, K, V values are $(m, d) \times (d, d)$.

Decode. We calculate one output token using the previous prompts' cached KV values.

Operations on hidden states x , such as calculating Q, K, V values are $(1, d) \times (d, d)$.

What Actually Gets Computed

Per layer, on hidden states X , with $W_Q \in \mathbb{R}^{d \times H_q h}$, $W_K, W_V \in \mathbb{R}^{d \times H_{kv} h}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{h}}\right) V$$

In prefill $X \in \mathbb{R}^{m \times d}$. In decode $X = x \in \mathbb{R}^{1 \times d}$, and K, V are the cached ones with the single new row appended.

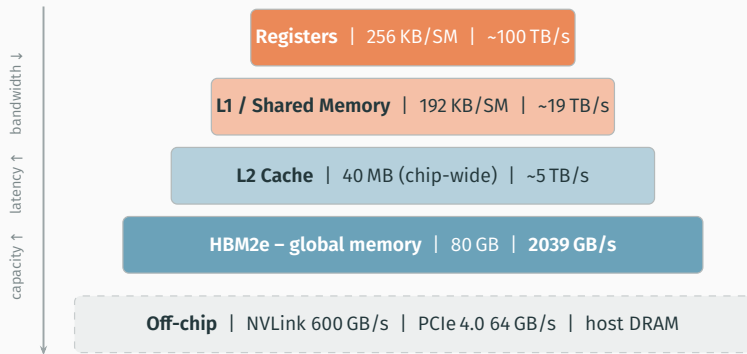
What Actually Gets Computed

Qwen3-4B: $d = 2560$, $L = 36$ layers, $H_q = 32$ query heads, $H_{kv} = 8$ KV heads, head dim $h = 128$, $d_{ff} = 9728$.

per layer	Prefill (m tokens)	Decode (1 token, cache m)
Q proj	$(m, d) \times (d, H_q h)$	$(1, d) \times (d, H_q h)$
K, V proj	$(m, d) \times (d, H_{kv} h)$	$(1, d) \times (d, H_{kv} h)$
scores QK	$(m, h) \times (h, m)$	$(1, h) \times (h, m)$
softmax $\cdot V$	$(m, m) \times (m, h)$	$(1, m) \times (m, h)$
MLP	$(m, d) \times (d, d_{ff})$	$(1, d) \times (d, d_{ff})$
FLOPs	$O(md^2 + m^2d)$	$O(d^2 + md)$

Problem: The cache removes the compute, but not the weight reads. Every matrix on the right still has to be pulled from HBM in full, to be multiplied against a single row.

The GPU Memory Hierarchy



Where reuse happens.

Data is pulled here and multiplied many times. Arithmetic intensity counts exactly this: FLOPs per byte lifted out of HBM.

Where the model lives.

8 GB of weights + the KV cache. Every decode step re-reads all of it.

Only the bottom level is big enough to hold the model, but it is very slow. For matrix multiplications, the weight matrices need to be pulled into higher GPU memory levels.

Roofline Model: Compute- vs Memory-Bound

A GPU operation is bound by whichever resource runs out first: **compute** (FLOPs/s) or **memory bandwidth** (bytes/s). The relevant quantity is **arithmetic intensity**:

$$AI = \frac{\text{FLOPs}}{\text{Bytes moved from HBM}} \quad \left[\frac{\text{FLOP}}{\text{byte}} \right]$$

Achieved throughput is capped by the **roofline**:

$$\text{Achieved FLOP/s} = \min(\text{Peak Compute}, AI \times \text{Memory Bandwidth})$$

The **ridge point** is where the two terms are equal — below it, you are memory-bound no matter how fast your cores are.

Reference GPU (A100 SXM 80GB, FP16 tensor cores):

$$\text{Peak Compute} = 312 \text{ TFLOP/s}, \quad \text{Bandwidth} = 2039 \text{ GB/s} \Rightarrow \text{Ridge} \approx 153 \frac{\text{FLOP}}{\text{byte}}$$

Any matmul with AI below 153 leaves compute cores idle, waiting on HBM.

Why Decoding is Slow

Take one linear projection of **Qwen3-4B**: $d = 2560$

	Decode $(1, d) \times (d, d)$	Prefill $(m=512, d) \times (d, d)$
FLOPs	$2d^2 \approx 13.1$ MFLOP	$2md^2 \approx 6.7$ GFLOP
Bytes moved (weights)	$2d^2 \approx 13.1$ MB	$2d^2 \approx 13.1$ MB
Arithmetic Intensity	1 FLOP/byte	512 FLOP/byte
Achieved throughput	2.0 TFLOP/s	312 TFLOP/s
GPU utilization	0.65%	100%

⇒ during decoding, the GPUs sit almost completely idle.

Batching can help, but only if you have multiple requests in parallel you can batch together.

Speculative Decoding

How to speed up our model (almost) for free

Idea. A cheap model (drafter) predicts tokens, original LLM (verifier) confirms tokens.

During decoding, drafter produces b proposal tokens $\hat{s}_{m+1}, \hat{s}_{m+2}, \dots, \hat{s}_{m+b}$. Verifier then batches the proposal tokens together to determine logits at each proposal position.

Where top-1 logit agrees with proposal: accept token, else reject. After the first rejected position, the full draft is thrown away $\implies$ the verifiers probability distribution is unaltered (lossless speedup).

Why does speculative decoding speed up inference?

Roofline argument: in memory-bound regime, processing M tokens is roughly as expensive as processing 1 token (until cross-over: so $M_{\text{cross}} \approx 153$ on the A100).

Per-round timing.

$$T_{\text{round}} = \gamma T_d + T_v$$

where T_v = verifier forward-pass time, T_d = drafter forward-pass time

Expected tokens per round. With per-position acceptance rate α_j :

$$E[\text{tokens/round}] = 1 + \sum_{i=1}^{\gamma} \prod_{j=1}^i \alpha_j \stackrel{\text{if } \alpha_j = \alpha \forall j}{=} \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

Speedup vs. plain autoregressive decoding (1 token every T_v):

$$\text{Speedup} = \frac{T_v}{\gamma T_d + T_v} \cdot \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

N-Gram Speculation

Idea. Take the last n tokens, search for the same n -gram earlier in prompt + generation, propose the γ tokens that followed it there. Verification is unchanged $\implies$ still lossless.

Cost. String lookup is almost free: $T_d \approx 0$.

$$\text{Speedup} = \frac{T_v}{\gamma T_d + T_v} \cdot \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \xrightarrow{T_d \rightarrow 0} \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

$\implies$ no interior optimum in γ any more; only the verifier batch approaching M_{cross} still limits it.

Where it helps. α is purely lexical: a draft is right only if the continuation *literally* stands in the context. Chat: $\alpha \approx 0$. Summarization, RAG, code edits: high.

Free: no second model, no weights in HBM, no shared tokenizer, no training.

Other Drafters

In principal, we can use any method that calculates new possible tokens given previous text and possibly verifier information like hidden states. Popular methods:

Drafter	T_d	α	needs training
standalone model	$0.1-0.2 T_v$	moderate	$\times$
EAGLE / Medusa head	$\ll T_v$	high	$\checkmark$
Diffusion LM	one pass for all γ	moderate	$\checkmark$
n -gram lookup	≈ 0	task-dependent	$\times$

EAGLE. Drafts from the verifier's hidden states instead of from text $\implies$ high α for one extra layer. Proposes a *tree* of continuations, verified in one pass.

Diffusion. All γ tokens in a single bidirectional pass: $\gamma T_d \rightarrow T_d$, so the term that created the interior optimum in γ disappears.

Quantization

How to get even faster, at a cost

Quantization

Quantization is a *lossy* speed-up technique: we perform matrix multiplications in lower precision.

Quantization does not always increase throughput. Some formats are supported by dedicated tensor cores on GPU, only these are performed faster. Else: very nuanced, faster memory transfer but slower execution (due to dequantization overhead).

Table 1: Quantization kernel support by NVIDIA GPU architecture

Architecture	INT8	FP8	FP4
Ampere	✓	✗	✗
Hopper	✓	✓	✗
Blackwell	(✓)	✓	✓

RTN: Uniform Affine Quantization

We approximate a float vector by **integers times a float scale**: $\hat{w} = s(w^{\text{int}} - z)$.

Asymmetric (scale s , zero point z), grid spans $[\min w, \max w]$:

$$s = \frac{\max(w) - \min(w)}{2^b - 1}, \quad z = \left\lfloor -\frac{\min(w)}{s} \right\rfloor, \quad w^{\text{int}} = \text{clamp}\left(\left\lfloor \frac{w}{s} \right\rfloor + z, 0, 2^b - 1\right)$$

Symmetric drops z , grid is centred on zero:

$$s = \frac{\max |w|}{2^{b-1} - 1}, \quad w^{\text{int}} = \text{clamp}\left(\left\lfloor \frac{w}{s} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right)$$

Round-to-nearest = do exactly this, no data, no search. One pass over the checkpoint.

Two error sources, traded against each other by s : **rounding** error $\leq s/2$ per weight, and **clipping** error for everything outside the grid. A single outlier inflates s and thereby degrades *every* weight sharing it.

Granularity: What Shares a Scale

The only real knob in RTN is how large a group shares one (s, z).

granularity	scales/matrix	extra bits/weight	robustness
per-tensor	1	≈ 0	worst
per-channel (per row)	d_{out}	≈ 0	better
group- g ($g=128$)	$d_{\text{out}} d_{\text{in}} / g$	$32/g = 0.25$	best

Effective bits = $b + 32/g$ (fp16 scale + zero point per group): 4-bit group-128 really costs 4.25 bit/weight.

Empirically. 8 bit is free at any granularity. 4 bit *needs* groups: per-tensor is broken, group-128 is within a few percent. 3 bit is the edge, 2 bit is gone.

⇒ every production 4-bit format (AWQ, GPTQ, NF4) is group-wise.

Dequantization: Where the Integers Turn Back Into Floats

Storing w^{int} is only half the story — the matmul has to consume it somehow.

W4A16 (AWQ, GPTQ). Weights 4-bit in HBM, activations fp16. No 4-bit tensor core exists, so the kernel **dequantizes in registers** and multiplies in fp16:

$$y = x\hat{W}^T, \quad \hat{W} = s \odot (W^{\text{int}} - z) \text{ unpacked on the fly}$$

Win: 4× fewer weight bytes. Cost: unpack + scale per weight, every matmul, every step.

W8A8 (INT8, FP8). Both operands low precision $\implies$ the tensor core does the matmul natively; dequantization is one multiply on the *output*, not on the weights.

Roofline consequence. W4A16 lifts decode AI from 1 to ≈ 4 FLOP/byte and adds dequant work. Batch 1: near-4×. Large batch: frequently *slower* than bf16.

Accumulation

A MAC array loads accumulators with the bias and adds products of the loaded operands:

$$\hat{A}_n = \hat{b}_n + \sum_m \hat{W}_{n,m} \hat{x}_m = \hat{b}_n + \sum_m (s_w W_{n,m}^{\text{int}})(s_x x_m^{\text{int}}) = \hat{b}_n + s_w s_x \sum_m W_{n,m}^{\text{int}} x_m^{\text{int}}$$

The scales **factor out of the sum** $\Rightarrow$ the inner loop is pure integer arithmetic.

With zero points the cross terms do not vanish:

$$\sum_m (W_{n,m}^{\text{int}} - z_w)(x_m^{\text{int}} - z_x) = \sum_m W_{n,m}^{\text{int}} x_m^{\text{int}} - \underbrace{z_w \sum_m x_m^{\text{int}}}_{\text{runtime}} - \underbrace{z_x \sum_m W_{n,m}^{\text{int}}}_{\text{offline}} + \underbrace{d_{\text{in}} z_w z_x}_{\text{offline}}$$

$\Rightarrow$ symmetric weights ($z_w = 0$) kill the one term that costs anything at runtime. This is why weight quantization in practice is symmetric even though it wastes a level.

Accumulator Width and Requantization

Products do not fit. $\text{INT8} \times \text{INT8}$ needs 16 bit; summing d_{in} of them needs $16 + \lceil \log_2 d_{\text{in}} \rceil$ bit — for $d = 2560$ that is 28 bit $\Rightarrow \text{INT32}$. Overflow is silent, so accumulators are always wider: $\text{INT8} \rightarrow \text{INT32}$, $\text{FP8} \rightarrow \text{FP32}$, $\text{fp16} \rightarrow \text{fp32}$.

Requantization. The INT32 accumulator must become an INT8 activation before the next layer, or you gained nothing on bandwidth:

$$x_{\text{next}}^{\text{int}} = \text{clamp} \left(\left\lfloor \frac{s_w s_x}{s_y} A_n^{\text{int}} \right\rfloor + z_y, 0, 255 \right)$$

with s_y from activation calibration. Fusing this into the matmul epilogue is what makes W8A8 fast.

Why W8A8 is not group-wise. Group scales break the factorization: s_w varies *inside* the sum, so the accumulator must be split per group and rescaled. Group-128 therefore lives in W4A16, where you dequantize anyway.

GPTQ: Rounding Is the Wrong Objective

RTN minimizes *weight* error $\|W - \hat{W}\|$. What we actually care about is *layer output* error on real data. With calibration activations $X \in \mathbb{R}^{d_{\text{in}} \times N}$:

$$\hat{W}^* = \arg \min_{\hat{W}} \|WX - \hat{W}X\|_2^2$$

Separable across output *rows* $\implies$ solve each row independently. *Not* separable across input columns: a direction X excites often deserves a smaller error. That coupling is exactly what RTN throws away.

Key quantity. The Hessian w.r.t. one row is the input covariance $H = 2XX^T \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}$ — one forward pass over ≈ 128 calibration sequences, accumulated per layer.

GPTQ: The Update

Quantize columns left to right. Having fixed column q to its grid value, the **optimal correction** to the remaining (still-unquantized) columns F is the OBS update:

$$e_q = \frac{w_q - \text{quant}(w_q)}{[H_F^{-1}]_{qq}}, \quad \delta_F = -e_q \cdot (H_F^{-1})_{q,F}$$

$\Rightarrow$ every rounding error is **immediately compensated** by nudging weights we have not committed to yet. Errors get absorbed instead of accumulating.

Why it is tractable. OBQ uses a per-row greedy order and re-inverts H each step: $O(d_{\text{in}}^4)$ per row. GPTQ fixes *one* column order for all rows $\Rightarrow$ a single Cholesky of H^{-1} , all rows updated as one matrix op.

Practicalities. Damping $H \leftarrow H + \lambda \bar{d}I$ ($\lambda \approx 1\%$), else the Cholesky fails. *Act-order*: columns by descending H_{qq} , so sensitive ones are quantized while correction budget remains. Lazy block updates (128 columns) to stay compute-bound.

Where This Lands

	data	cost	3–4 bit	8 bit
RTN	none	seconds	breaks	fine
GPTQ	≈ 128 seqs	minutes–hours, $O(d^3)$ /layer	good	fine
AWQ	≈ 128 seqs	minutes	good	fine

AWQ, in one line. Instead of correcting rounding errors, *rescale* channels before rounding: $\hat{W} = Q(W \cdot \text{diag}(\alpha))$, $\hat{x} = \text{diag}(\alpha)^{-1}x$, with α chosen from activation magnitudes so salient channels land on a finer effective grid. Mathematically free at inference — α folds into the previous layer.

$\Rightarrow$ all three are *indistinguishable* at 8 bit. They only separate in the 3–4 bit range, which is precisely where a 2× over FP8 is on the table.

Questions?